



LTM ETL PIPELINE

Development & Testing Documentation

This document records the design, implementation and validation of the LTM (Learning & Training Management) ETL pipeline, built end-to-end across Landing, Staging and Base layers with SCD Type 2 history tracking, as part of the Data Engineering internship at Accenture.

Prepared by	Ikjot Kour
Employee ID	14184237
Role	Data Engineering Intern, Accenture
Project	LTM ETL Pipeline (Learning & Training Management)
Duration	18 th May 2026 – 10 th July 2026

TABLE OF CONTENTS

1. Overview	3
1.1 Design Rules	3
1.2 Implementation Evidence	3
1.3 Load Control & Audit Framework.....	3
2. Landing Layer	5
2.1 Design Rules	5
2.2 Implementation Evidence	5
3. Staging Layer	8
3.1 Design Rules	8
3.2 Implementation Evidence	8
4. Base Layer	13
4.1 Design Rules	13
4.2 Implementation Evidence	13
5. Testing	17
5.1 Landing Layer Test Results	17
5.2 Staging Layer Test Results	17
5.3 Base Layer Test Results	18
5.4 Cross-Layer Testing	18
6. Views & Power BI Analysis	19
6.1 Reporting View.....	19
6.2 Power BI Analysis	19
7. Conclusion	20

1. Overview

This document records the design, implementation and validation of the LTM (Learning & Training Management) ETL pipeline, built as part of the Data Engineering internship at Accenture. The pipeline follows a three-layer Three stage pipeline architecture — Landing, Staging, and Base — with SCD Type 2 history maintained in the Base layer. Each layer's section below covers its design rules, implementation evidence, and test results.

1.1 Pipeline Architecture

The pipeline is organized into three sequential layers. Data is first extracted from multiple source files, loaded as-is into Landing; Staging then cleans, validates, enriches and transforms the data; and the Base layer stores the integrated, versioned data used for reporting and analytics.

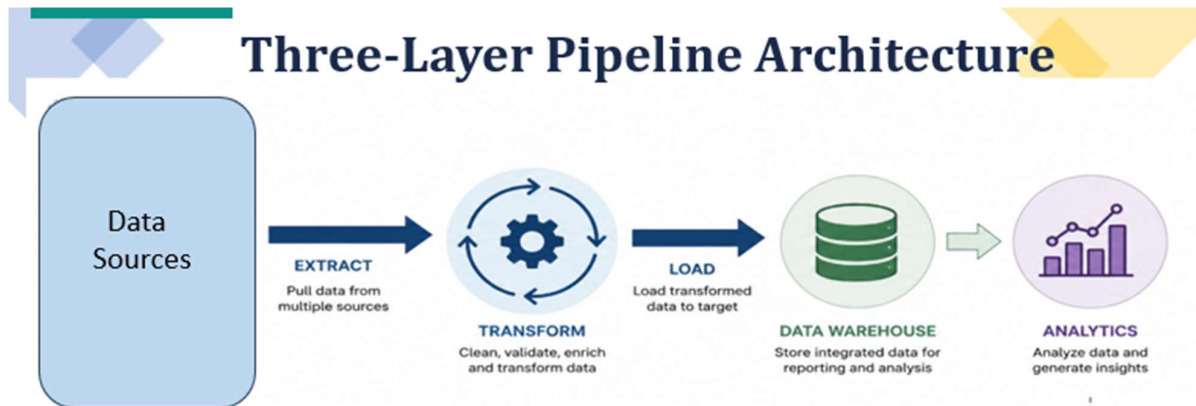


Fig 0.1 — Three-layer pipeline architecture: Extract (Landing) → Transform (Staging) → Load (Base/Data Warehouse) → Analytics.

1.2 Data Model (Entity-Relationship Diagram)

The pipeline processes five related entities. Employee is the core dimension; Enrollment and Assessment link an Employee to a Course (many-to-many, resolved through these two fact-like tables); and Trainer links a trainer record to the Course they deliver. This ER diagram defines the primary/foreign key relationships enforced across Landing, Staging and Base.

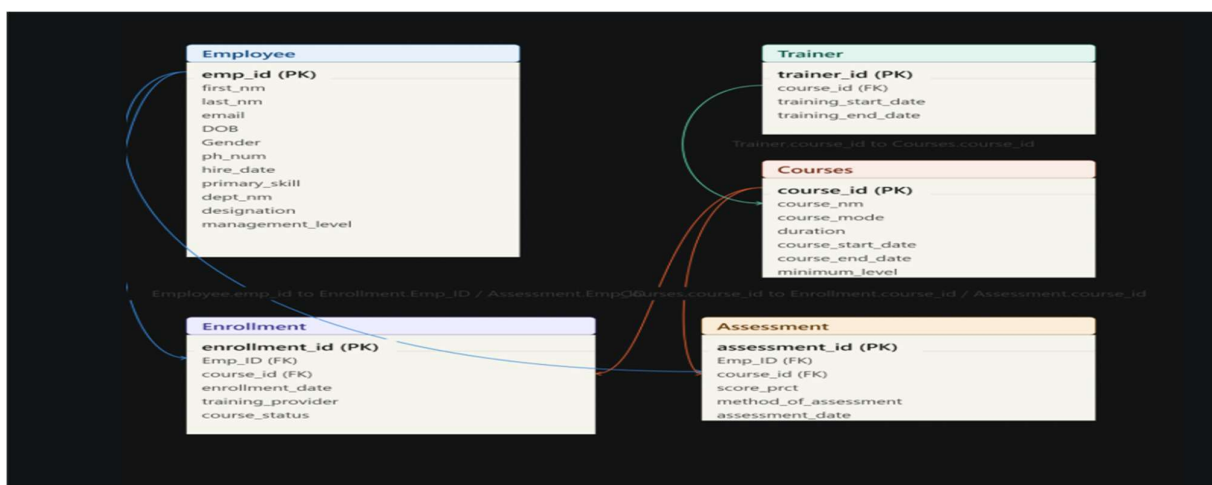


Fig 0.2 — Entity-relationship diagram: Employee, Trainer, Courses, Enrollment and Assessment, with primary and foreign key relationships.

1.3 Load Control & Audit Framework

To ensure complete traceability and monitoring of the ETL pipeline, two metadata tables are maintained throughout every ETL execution.

Load Control Table

The Load Control table records one entry for every ETL execution. Each run generates a unique Load Key, which is shared across all pipeline stages and associated with every record processed during that execution. It stores the project name, execution timestamp, pipeline status, and acts as the unique identifier for the ETL run.

control_id	project_name	load_key	load_date	status
1	LTM	LCK_20260707_231149_XAJIOY	2026-07-07 23:11:50	success
2	LTM	LCK_20260707_231234_XAJIOY	2026-07-07 23:12:35	success
3	LTM	LCK_20260707_231304_XAJIOY	2026-07-07 23:13:05	success
4	LTM	LCK_20260707_231334_XAJIOY	2026-07-07 23:13:35	success

Figure 1 load control table

Audit Table

The Audit Table captures execution details for every ETL stage, including Landing, Staging, and Base. Every processing step inserts one audit record containing:

- Target table name
- Script name
- Load Key
- Processing timestamp
- Processing status
- Source file path (Landing layer only)
- Number of processed rows

Together, the Load Control and Audit tables provide complete end-to-end monitoring, execution traceability, debugging support, and operational reporting for the ETL pipeline.

audit_id	table_name	file_name	ingest_timestamp	load_key	status	file_path	row_count
15	base_assessment	base_extract.py	2026-07-07 23:12:17	LCK_20260707_231149_XAJIOY	success	NULL	282
16	landing_employee	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source\emp_120525.csv	3
17	landing_course	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source\course_130525.csv	50
18	landing_trainer	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source\trainer_110525.csv	9
19	landing_enrollment	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source\enrollment_130525.csv	784
20	landing_assessment	landing_extract.py	2026-07-07 23:12:35	LCK_20260707_231234_XAJIOY	success	source\assessment_130525.csv	276
21	staging_employee	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	NULL	3
22	staging_course	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	NULL	50
23	staging_trainer	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	NULL	7
24	staging_enrollment	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	NULL	784
25	staging_assessment	staging_extract.py	2026-07-07 23:12:42	LCK_20260707_231234_XAJIOY	success	NULL	273
26	base_employee	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	NULL	3
27	base_course	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	NULL	50
28	base_trainer	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	NULL	7
29	base_enrollment	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	NULL	784
30	base_assessment	base_extract.py	2026-07-07 23:12:51	LCK_20260707_231234_XAJIOY	success	NULL	273
31	landing_employee	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	already_processed	source\emp_130525.csv	0
32	landing_course	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	no_file_found	NULL	0
33	landing_trainer	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	already_processed	source\trainer_120525.csv	0
34	landing_enrollment	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	already_processed	source\enrollment_140525.csv	0
35	landing_assessment	landing_extract.py	2026-07-07 23:13:05	LCK_20260707_231304_XAJIOY	already_processed	source\assessment_140525.csv	0
36	landing_employee	landing_extract.py	2026-07-07 23:13:35	LCK_20260707_231334_XAJIOY	success	source\emp_140525.csv	3
37	landing_course	landing_extract.py	2026-07-07 23:13:35	LCK_20260707_231334_XAJIOY	no_file_found	NULL	0

Figure 2 Audit Table

2. Landing Layer

2.1 Design Rules

- Skip if missing: If no source file exists for a particular entity, the process skips that entity and continues with the remaining files.
- Empty file handling: If an empty file is encountered, it is discarded, logged in the audit table with an EMPTY_FILE / failed status, and is not loaded into the landing tables.
- Supported formats: The pipeline correctly processes CSV, Excel (.xlsx) and JSON files. Unsupported file formats are rejected, logged as failures, and skipped without interrupting the ETL run.
- Chronological processing: Files are processed in ascending date order (oldest first), based on the date embedded in the filename, to maintain chronological consistency.
- Each Landing execution generates a Load Key and records processing information in the centralized Load Control and Audit framework described in Section 1.3.
- Raw preservation: During landing, raw data is preserved without business transformations. Only metadata — Load Key and Ingestion Timestamp — is appended.
- Archival: After successful processing, source files are moved to the Archive folder, where only the latest seven files per entity are retained.
- Idempotent re-runs: Once a source file has been successfully processed and archived, re-running the pipeline detects this and skips the file rather than reloading it, so landing rows are never duplicated across runs.
- **Audit Entry:** Records the Landing execution in the Audit Table with the target table, script name, load key, processing timestamp, status, source file path, and rows processed.

2.2 Implementation Evidence

2.2.1 Skip Entity When No File Exists

When a file is not found for an entity (e.g. Course), the pipeline logs a skip message and continues processing the remaining entities without failing the run.

```
PS C:\Users\ikjot.kour\OneDrive - Accenture\Desktop\ETL_Project> python landing_extract.py
processing emp_130525.csv for table employee...
no file found for course, skipping
processing trainer_120525.csv for table trainer...
processing enrollment_140525.csv for table enrollment...
processing assessment_140525.csv for table assessment...

landing step done
```

Fig 1 — landing_extract.py skipping Course (no file found) and continuing with Trainer, Enrollment and Assessment.

2.2.2 Empty File Handling

Files with no parsable data raise a handled error during processing and are excluded from the landing load rather than crashing the pipeline.

```
PS C:\Users\ikjot.kour\OneDrive - Accenture\Desktop\ETL_Project> python landing_extract.py
no file found for employee, skipping
no file found for course, skipping
processing trainer_100525.csv for table trainer...
error processing trainer_100525.csv: No columns to parse from file
no file found for enrollment, skipping
no file found for assessment, skipping

landing step done
PS C:\Users\ikjot.kour\OneDrive - Accenture\Desktop\ETL_Project> █
```

Fig 2 — `trainer_100525.csv` fails with “No columns to parse from file”; the pipeline logs the error and continues to the next entity.

2.2.3 Unsupported File Type Rejection

Non-CSV/XLSX/JSON files, such as a .md file, are correctly identified and rejected as an unsupported file type.

```
processing trainer_100525.md for table trainer...
error processing trainer_100525.md: unsupported file type: source\trainer_100525.md
```

Fig 3 — `trainer_100525.md` rejected with “unsupported file type” error.

2.2.4 Chronological (Ascending Date) Processing

Source files for each entity are picked up oldest-first based on the date embedded in the filename, keeping load order consistent across entities.

```
PS C:\Users\ikjot.kour\OneDrive - Accenture\Desktop\ETL_Project> python landing_extract.py
processing emp_120525.csv for table employee...
processing course_130525.csv for table course...
processing trainer_110525.csv for table trainer...
processing enrollment_130525.csv for table enrollment...
processing assessment_130525.csv for table assessment...
```

Fig 4 — `emp_120525`, `course_130525`, `trainer_110525`, `enrollment_130525` and `assessment_130525` processed in ascending date order.

2.2.5 Raw Data Preservation with Metadata Append

Landing tables retain the source data as-is (e.g. Assessment records with score, method, and date exactly as sourced) with only Ingest Timestamp and Load Key appended as new metadata columns.

assessment_id	Emp_ID	course_id	score_prct	method_of_assessment	assessment_date	ingest_timestamp	load_key
A00001	E00013	C00050	71	Hands-on	2021-08-19	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00002	E00017	C00033	84	Applied Project Simulation	2023-03-28	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00003	E00018	C00029	41	Hands-on	2026-02-09	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00004	E00018	C00009	60	Hands-on	2021-12-23	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00005	E00019	C00025	60	Hands-on	2023-01-24	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00006	E00021	C00027	99	Applied Project Simulation	2024-10-27	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00007	E00023	C00025	57	Hands-on	2023-11-09	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00008	E00023	C00031	68	Hands-on	2022-08-06	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00009	E00024	C00041	41	Applied Project Simulation	2024-07-05	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00010	E00025	C00033	85	Applied Project Simulation	2023-05-22	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00011	E00027	C00032	78	Client Interview	2021-06-07	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00012	E00028	C00024	77	Competency	2022-09-20	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00013	E00029	C00010	84	Competency	2026-04-17	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00014	E00030	C00014	67	Competency	2022-08-11	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00015	E00031	C00029	62	Hands-on	2025-07-17	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00016	E00039	C00045	86	Client Interview	2023-10-14	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00017	E00041	C00043	76	Applied Project Simulation	2025-04-17	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y
A00018	E00041	C00047	76	Client Interview	2025-08-15	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y

Fig 8 — `landing_assessment` table: raw fields preserved, with `ingest_timestamp` and `load_key` appended.

2.2.6 Source File Inventory

The source folder contains a mix of valid (CSV/XLSX/JSON) and invalid (.md) files across multiple dates per entity, used to exercise the format-validation and chronological-ordering logic shown above.

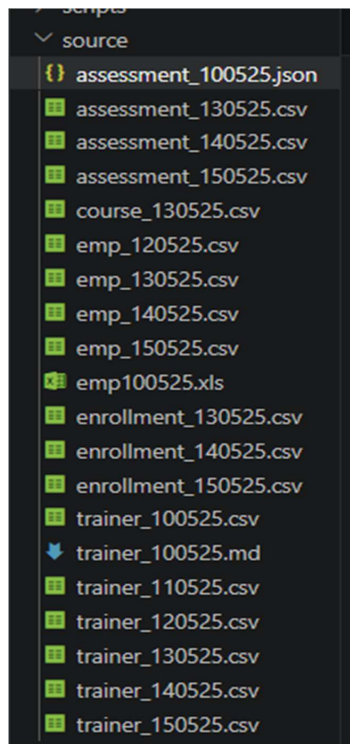


Fig 9 — source folder listing showing CSV, XLSX, JSON and an unsupported .md file across dated batches.

2.2.7 Idempotent Skip of Already-Processed Files

Re-running the extract against a source folder that has already been ingested does not create duplicate landing rows. Each file is checked against prior processing history and, if it was already loaded and archived, it is skipped with a log message instead of being re-read; only files that have not yet been processed continue on to Landing and the Archive folder.

```
PS C:\Users\ikjot.kour\OneDrive - Accenture\Desktop\ETL_Project> python landing_extract.py
processing emp_120525.csv for table employee...
emp_120525.csv already processed. Skipping.
processing course_130525.csv for table course...
course_130525.csv already processed. Skipping.
processing trainer_110525.csv for table trainer...
trainer_110525.csv already processed. Skipping.
processing enrollment_130525.csv for table enrollment...
enrollment_130525.csv already processed. Skipping.
processing assessment_130525.csv for table assessment...
assessment_130525.csv already processed. Skipping.

landing step done
PS C:\Users\ikjot.kour\OneDrive - Accenture\Desktop\ETL_Project>
```

Fig 9.1 — landing_extract.py re-run: emp_120525.csv, course_130525.csv, trainer_110525.csv, enrollment_130525.csv and assessment_130525.csv are all reported as “already processed. Skipping.”, confirming the archive-based idempotency check.

3. Staging Layer

3.1 Design Rules

- Full transfer: All records from the Landing tables are successfully transferred to the corresponding Staging tables.
- Mandatory field validation: Mandatory fields (IDs and required business attributes) are validated before further processing.
- Duplicate identification: Duplicate records are identified using business keys and checksums to prevent unnecessary updates.
- Checksum generation: A checksum is generated for every record to detect changes between the Landing and Base layers.
- Change flags are assigned correctly:
 - I — New record
 - U — Existing record with modified data
 - N — Existing record with no changes
- SCD Type 2 logic:
 - New records are inserted with Current_Ind = 'Y'.
 - Modified records create a new version while the previous version is marked inactive (Current_Ind = 'N').
 - The previous record's End_Date is updated to New Start_Date – 1.
- Date validation: Invalid dates are converted to NULL; records with inconsistent date ranges (Start_Date > End_Date) are identified and handled appropriately.
- Data standardization: Employee names are capitalized, gender values are standardized (Male/Female/M/F → common format), and date formats are converted to a consistent standard.
- Business rule validation: Business rule validations are executed before loading to the Base layer; records that fail validation are excluded, reducing unnecessary overhead in Base.
- Selective insertion: Only records with change flags I and U are eligible for insertion into the Base layer; unchanged (N) records are not reloaded.
- Post-load truncation: After successful loading into the Base layer, the Staging tables are truncated to free storage and prepare for the next ETL cycle.
- **Audit Logging:** Records the execution details in the centralized Audit Table for end-to-end ETL traceability.

3.2 Implementation Evidence

3.2.1 Staging Run Overview

Running staging_extract.py pulls all Landing data into Staging, computes a checksum per record, and applies SCD Type 2 comparison logic against Base for every entity — reporting counts of new (I), changed (U), unchanged (N), and skipped duplicate records.

```

landing step done
PS C:\Users\ikjot.kour\OneDrive - Accenture\Desktop\ETL_Project> python staging_extract.py
staging for load_key: LCK_20260705_135540_XAJI0Y
staging timestamp: 2026-07-05 13:55:55.207628

processing employee...
  flagged I (new): 1  flagged U (changed): 1  skipped N (no change): 1  skipped (duplicate in staging): 0
processing course...
  flagged I (new): 0  flagged U (changed): 0  skipped N (no change): 50  skipped (duplicate in staging): 0
processing trainer...
  invalid date range: training_start_date=2026-07-31 > training_end_date=2026-06-04 - nulled
  flagged I (new): 0  flagged U (changed): 1  skipped N (no change): 8  skipped (duplicate in staging): 0
processing enrollment...
  flagged I (new): 784  flagged U (changed): 0  skipped N (no change): 0  skipped (duplicate in staging): 0
processing assessment...
  flagged I (new): 275  flagged U (changed): 0  skipped N (no change): 0  skipped (duplicate in staging): 0

staging step done

```

Fig 10 — staging_extract.py run summary per entity, including a nulled invalid date range for Trainer.

3.2.2 Full Transfer Verification (Row Count)

On a first load, the Staging row count matches the Landing row count exactly, confirming no records are lost in transfer.

```

mysql> select count(*) from staging_assessment;
+-----+
| count(*) |
+-----+
|      275 |
+-----+
1 row in set (0.065 sec)

mysql> select count(*) from landing_assessment;
+-----+
| count(*) |
+-----+
|      275 |
+-----+
1 row in set (0.048 sec)

```

Fig 11 — staging_assessment and landing_assessment both return 275 rows, confirming a full first-load transfer.

3.2.3 Checksum and Change-Flag Assignment

Every staged record carries a computed checksum and a change_flag. On first load, all records are correctly flagged 'I' (new insertion).

assessment_key	assessment_id	emp_id	course_id	score_pct	method_of_assessment	assessment_date	ingest_timestamp	load_key	checksum	change_flag
A00001_20260705135555	A00001	E00013	C00050	71	Hands-on	2021-08-19	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	594849e9440881c73d8d6f7a4801b418	I
A00002_20260705135555	A00002	E00017	C00033	84	Applied Project Simulation	2023-03-28	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	bb470c074a2f6b2bf4c1369853c5922	I
A00003_20260705135555	A00003	E00018	C00029	41	Hands-on	2026-09-02	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	773bcad1b3331685fe3d35de44480551	I
A00004_20260705135555	A00004	E00018	C00009	60	Hands-on	2021-12-23	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	e5e4837565d9a73fb711bf69ed314a2d	I
A00005_20260705135555	A00005	E00019	C00025	60	Hands-on	2023-01-24	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	bacc62d4fafd6830b4c42db97c5915bdb	I
A00006_20260705135555	A00006	E00021	C00027	99	Applied Project Simulation	2024-10-27	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	67654c649c02f733f46a537bc0d0ffa	I
A00007_20260705135555	A00007	E00023	C00025	57	Hands-on	2023-09-11	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	2046a00406cc215685b2935aee111cc4	I
A00008_20260705135555	A00008	E00023	C00031	68	Hands-on	2022-06-08	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	a1538f4c79a8d46f63680a67ae366e	I
A00009_20260705135555	A00009	E00024	C00041	41	Applied Project Simulation	2024-05-07	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	cfb04fb4340c2e33011979eaa736f417	I
A00010_20260705135555	A00010	E00025	C00033	85	Applied Project Simulation	2023-05-22	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	83869f6cah250846844a7aa95be1870	I
A00011_20260705135555	A00011	E00027	C00032	78	Client Interview	2021-07-06	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	83bfc4c54f05cb7862e223563c06a2241	I
A00012_20260705135555	A00012	E00028	C00024	77	Competency	2022-09-20	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	b862bd7fca083a46f5939b6c3e45178	I
A00013_20260705135555	A00013	E00029	C00010	84	Competency	2026-04-17	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	bed37082d01f5d6538f33428af3491c9	I
A00014_20260705135555	A00014	E00030	C00014	67	Competency	2022-11-08	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	e069cf509594af7677af0c2e1941f7b	I
A00015_20260705135555	A00015	E00031	C00029	62	Hands-on	2025-07-17	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	bac7bc2d036e8c7378e0c096c61c0beb	I
A00016_20260705135555	A00016	E00039	C00045	86	Client Interview	2023-10-14	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	b97711d9efb94c4ae5d9b093b2d5c55	I
A00017_20260705135555	A00017	E00041	C00043	76	Applied Project Simulation	2025-04-17	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	75435ffbd4e4285a45d58aa7e9aa394	I
A00018_20260705135555	A00018	E00041	C00047	76	Client Interview	2025-08-15	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	1a70c3e60895e79401742dbe207b5479c	I
A00019_20260705135555	A00019	E00046	C00031	41	Competency	2022-08-04	2026-07-05 13:55:41	LCK_20260705_135540_XAJI0Y	492feb7bc4339a1200ab7ff3d8f007a5	I

Fig 12 — staging_assessment table with checksum and change_flag columns, all flagged 'I' on first load.

3.2.4 Date Validation

Rows with an invalid date range (Start_Date > End_Date) are identified and have their dates nulled rather than being loaded as-is.

T00208_20260623150702	T00208	C00007	2026-05-31	2026-06-04	2026-06-23 15:06:56	LCK_20260623_150655_XAJI0Y	180353c2dd58b9923874fc5ad0b17863	2026-06-23	2026-06-25	N
T00208_20260626131808	T00208	C00007	NULL	NULL	2026-06-26 13:17:59	LCK_20260626_131758_XAJI0Y	e36fc55d1aeed31c80105266fca01bf5	2026-06-26	9999-12-31	Y
T00209_20260623150702	T00209	C00027	2021-10-17	2021-10-22	2026-06-23 15:06:56	LCK_20260623_150655_XAJI0Y	7e8a623fc6429fca19f806fb709074b	2026-06-23	9999-12-31	Y

Fig 13 — sample staging rows where an invalid date range has been nulled.

```

SKIPPP: no data found in landing_course
processing trainer...
  invalid date range: training_start_date=2025-03-15 > training_end_date=2025-01-28 - nulled
  invalid date range: training_start_date=2025-05-04 > training_end_date=2025-04-15 - nulled
  flagged I (new): 0  flagged U (changed): 6  skipped N (no change): 2  skipped (duplicate in staging): 0
processing enrollment...
  flagged I (new): 818  flagged U (changed): 0  skipped N (no change): 0  skipped (duplicate in staging): 0
processing assessment...
  flagged I (new): 282  flagged U (changed): 0  skipped N (no change): 0  skipped (duplicate in staging): 0

```

Fig 14 — staging_extract.py run showing a Course entity skip (no Landing data) alongside nulled invalid Trainer date ranges.

3.2.5 Transformation Logic

The apply_transformations() function standardizes casing, renames non-standard columns, cleans whitespace, handles nulls, capitalizes names, standardizes dates and gender values, and validates email, phone and date-range fields before a record is eligible for Base.

```

def apply_transformations(row, table_name):
    row = standardize_column_casing(row)
    row = rename_columns(row)           # CHANGE 1: rename non-standard column names
    clean_whitespace(row)
    handle_nulls(row)
    capitalize_name_columns(row)       # CHANGE 2: now catches _nm columns too
    standardize_dates(row)
    validate_date_range(row, table_name)
    validate_email(row)
    validate_phone(row)
    standardize_gender(row)
    if table_name == "assessment":
        validate_assessment_score(row)
    return row

```

Fig 15 — apply_transformations() showing the full sequence of validation and standardization steps.

3.2.6 Data Standardization — Before and After

Comparing raw Landing data against the staged output confirms names are capitalized and gender values are normalized to a consistent M/F format.

```
emp_id,first_nm,last_nm,email,D0B,Gender,ph_num,hire_date,primary_sl
E00001,Nathaniel,Taylor,jacobsuarez@example.net,1986-07-17,Male,9876
E00002, robert ,Moore,diana95@example.net,1999-09-25,male,98765012
E00003,Diana,montoya,raymondjones@example.com,1995-05-10,Female,9187
```

Fig 16 — raw employee data as received in Landing (inconsistent casing, e.g. “robert”, “male”).

E00002_20260705135555	E00002	Robert	Moore	diana95@example.net	1999-09-25	M
E00003_20260705135555	E00003	Diana	Montoya	raymondjones@example.com	1995-10-05	F
E00004_20260703125907	E00004	Ariana	Grande	raymondjones@example.com	1995-10-05	F
E01001_20260703130012	E01001	Erica	Woods	eharris@example.org	1980-09-23	F
E01002_20260703130012	E01002	Vernon	Harrington	morankenneth@example.net	1999-05-20	M

Fig 17 — same employee records after Staging standardization: names capitalized, gender normalized to M/F.

3.2.7 Post Base-Load Truncation

Once records are successfully loaded into Base, the Staging tables are truncated — confirmed by staging_assessment returning 0 rows while base_assessment holds the loaded records.

```
mysql> select count(*) from staging_assessment;
+-----+
| count(*) |
+-----+
|         0 |
+-----+
1 row in set (0.006 sec)

mysql> select count(*) from base_assessment;
+-----+
| count(*) |
+-----+
|        555 |
+-----+
1 row in set (0.020 sec)
```

Fig 18 — staging_assessment truncated to 0 rows; base_assessment holds 555 loaded records.

3.2.8 Mandatory Field Validation

Records missing a mandatory identifier — such as a null assessment_id — are detected during staging and excluded from further processing rather than being carried forward with an incomplete key.

```
processing assessment...
Skipping record - mandatory field missing: {'assessment_id': None, 'emp_id': 'E00013', 'course_id': 'C00050', 'score_prct': 71, 'method_of_assessment': '
Hands-on', 'assessment_date': '2021-08-19', 'ingest_timestamp': datetime.datetime(2026, 7, 5, 15, 12, 29), 'load_key': 'LCK_20260705_151228_XAJI0Y'}
Skipping record - mandatory field missing: {'assessment_id': None, 'emp_id': 'E00017', 'course_id': 'C00033', 'score_prct': 84, 'method_of_assessment': '
Applied Project Simulation', 'assessment_date': '2023-03-28', 'ingest_timestamp': datetime.datetime(2026, 7, 5, 15, 12, 29), 'load_key': 'LCK_20260705_15
1228_XAJI0Y'}
```

Fig 18.1 — staging_extract.py log: two Assessment records with assessment_id = None (Emp E00013 / course C00050 and Emp E00017 / course C00033) are skipped with a “mandatory field missing” warning before checksum and change-flag logic runs.

The underlying raw rows show the gap directly: the same two records return assessment_id = nan while every other row in the batch carries a valid, populated ID.

assessment_id	Emp_ID	course_id	score_pct	method_of_assessment	assessment_date	ingest_timestamp	load_key
nan	E00013	C00050	71	Hands-on	2021-08-19	2026-07-05 15:06:08	LCK_20260705_150607_XAJI0Y
nan	E00017	C00033	84	Applied Project Simulation	2023-03-28	2026-07-05 15:06:08	LCK_20260705_150607_XAJI0Y
A00003	E00018	C00029	41	Hands-on	2026-02-09	2026-07-05 15:06:08	LCK_20260705_150607_XAJI0Y
A00004	E00018	C00009	60	Hands-on	2021-12-23	2026-07-05 15:06:08	LCK_20260705_150607_XAJI0Y

Fig 18.2 — pre-validation query showing assessment_id = nan for the Emp E00013 and Emp E00017 rows, alongside correctly populated IDs such as A00003 and A00004 in the same result set.

Once the mandatory-field check is applied, only records with a valid key move forward into Staging; every surviving row carries a computed checksum and the load_key of the run that staged it.

assessment_key	assessment_id	emp_id	course_id	score_pct	method_of_assessment	assessment_date	ingest_timestamp	load_key	checksum
A00003_20260705151235	A00003	E00018	C00029	41	Hands-on	2026-09-02	2026-07-05 15:12:29	LCK_20260705_151228_XAJI0Y	773bcad1b3331685fe3d35de44400551
A00004_20260705151235	A00004	E00018	C00009	60	Hands-on	2021-12-23	2026-07-05 15:12:29	LCK_20260705_151228_XAJI0Y	e5e4037565d9a73fb7a1bf69ed314e2d
A00005_20260705151235	A00005	E00019	C00025	60	Hands-on	2023-01-24	2026-07-05 15:12:29	LCK_20260705_151228_XAJI0Y	bacc62dfaf6d830b4c42db97c5915bdb
A00006_20260705151235	A00006	E00021	C00027	99	Applied Project Simulation	2024-10-27	2026-07-05 15:12:29	LCK_20260705_151228_XAJI0Y	67654c649cd2f733f46a4537bcb0dffa
A00007_20260705151235	A00007	E00023	C00025	57	Hands-on	2023-09-11	2026-07-05 15:12:29	LCK_20260705_151228_XAJI0Y	2846b00460cc215605b2935aec111cc4
A00008_20260705151235	A00008	E00023	C00031	68	Hands-on	2022-06-08	2026-07-05 15:12:29	LCK_20260705_151228_XAJI0Y	a1538f4c79e8d64f6368d0a6f7ae3e6e
A00009_20260705151235	A00009	E00024	C00041	41	Applied Project Simulation	2024-05-07	2026-07-05 15:12:29	LCK_20260705_151228_XAJI0Y	cf0b4fb4340c2e33011979eaa736f417
A00010_20260705151235	A00010	E00025	C00033	85	Applied Project Simulation	2023-05-22	2026-07-05 15:12:29	LCK_20260705_151228_XAJI0Y	83869f6cab2508d46844d7aa95be8170

Fig 18.3 — staging_assessment after validation: assessment_key, checksum and load_key populated for every remaining record (A00003–A00010), confirming records with missing mandatory fields never reach this stage.

4. Base Layer

4.1 Design Rules

- Base table structure: For every entity, a base_<table> table is created (if it does not already exist) with the business key as primary key, one column per staged attribute, plus checksum, start_date, end_date and curr_ind columns.
- Change-flag driven processing: Base only acts on Staging rows carrying change_flag I or U. Rows flagged N are skipped as a safety net; Staging should never forward these.
- New record insertion (I): A brand-new business key is inserted directly, with start_date = today, end_date = 9999-12-31 (open-ended) and curr_ind = 'Y'.
- SCD Type 2 versioning (U): The currently active row (curr_ind = 'Y') for that key is expired — curr_ind set to 'N' and end_date set to today minus 1 day — and a new row is inserted with start_date = today, end_date = 9999-12-31 and curr_ind = 'Y', so history never overlaps.
- Single active-version rule: At most one row per business key may have curr_ind = 'Y' at any time; the old version is always expired before the new version is inserted.
- Per-entity error isolation: Employee, Course, Trainer, Enrollment and Assessment are each processed and committed independently. If one entity's batch fails, its error is logged and its Staging table is left un-truncated, while the remaining entities continue processing normally in the same run.
- Truncate-after-success only: A Staging table is cleared only once its corresponding Base load for that run has completed without error.
- Run-level counts: Each run reports inserted / updated / skipped counts per entity, giving a traceable summary of what changed in Base.
- Expiry convention: end_date = 9999-12-31 is the sentinel 'last possible date', used to mark a row as currently open-ended/active (curr_ind = 'Y'); a real, past end_date paired with curr_ind = 'N' marks a row that has been expired by a later version.
- **Audit Logging:** Records the execution details in the centralized Audit Table for end-to-end ETL traceability.

4.2 Implementation Evidence

4.2.0 Base Layer Run Overview

Running base_extract.py applies the change-flag logic per entity: change_flag = I rows are inserted, change_flag = U rows trigger an SCD Type 2 expiry-and-insert, and change_flag = N rows are left untouched (reported as skipped). Each entity's Staging table is cleared only after its Base load for that run completes successfully.

```

PS C:\Users\ikjot.kour\OneDrive - Accenture\Desktop\ETL_Project> python base_extract.py
base layer execution: 2026-07-05 16:28:09.906671

processing employee...
  inserted: 497 updated: 3 skipped: 0
  staging_employee cleared
processing course...
  SKIPPP!! no data found in staging_course
processing trainer...
  inserted: 3 updated: 3 skipped: 0
  staging_trainer cleared
processing enrollment...
  inserted: 818 updated: 0 skipped: 0
  staging_enrollment cleared
processing assessment...
  inserted: 282 updated: 0 skipped: 0
  staging_assessment cleared

base layer run done

```

Fig 19 — `base_extract.py` run: Employee (497 inserted, 3 updated, 0 skipped), Trainer (3 inserted, 3 updated, 0 skipped), Enrollment (818 inserted) and Assessment (282 inserted) — each followed by its corresponding staging_<entity> table being cleared. Course is skipped as no data was staged for it in this run.

4.2.1 New Record Insertion (Change Flag I)

On first load, every business key arriving from Staging is new, so each row is inserted directly into the Base table with `curr_ind = 'Y'`, `start_date = today` and `end_date = 9999-12-31`. Comparing the row count in Landing against the row count in Base for the same entity confirms every record was inserted exactly once, with no duplicates introduced during the load.

```

mysql> select distinct count(*) from landing_trainer;
+-----+
| count(*) |
+-----+
|          9 |
+-----+
1 row in set (0.025 sec)

mysql> select distinct count(*) from base_trainer;
+-----+
| count(*) |
+-----+
|          9 |
+-----+
1 row in set (0.022 sec)

```

Fig 20 — `landing_trainer` and `base_trainer` both return a distinct count of 9 rows — the base load carried every landing record across exactly once, with no duplicate insertion.

Every one of these newly staged records also carries `change_flag = 'I'`, consistent with a first load where nothing yet exists in Base to compare against.

checksum	change_flag
93b04775e4764e97f323afd10ee7d81d	I
b236f5421c1c0c48a3d5f6104ba3722c	I
cbfad10409c553845b44c2a597a5da05	I
7430a4ad028fe0864d5f9d3be2313060	I
82ce1e57eee5d66686bd0fbde6341739	I
9977482077f157f8fb61c2b9ec1241a1	I
f380864287b862dbfbac8bce0eb4518f	I
2eea74c62691e062355bd71d07386d0c	I
7abfab78700049a82a1da12dd52bb6b9	I
bbe2f98e13bb95683a811aed795db186	I
7687f4dd5c3aec93a61be92bbe59ec28	I
af0a602f590d4e1c432a836b2514d0cb	I
239cc406f88e154aeb72ea90975b5f44	I
294761a0b603ca388149468bdce75568	I

Fig 21 — staging output with computed checksum and change_flag columns: every row is flagged 'I', confirming these are all new records eligible for straight insertion into Base.

4.2.2 SCD Type 2 Update (Change Flag U)

When a business key reappears with change_flag = U, the previously active row is expired (curr_ind → 'N', end_date → new start_date minus 1 day) and a new row is inserted as the current version (curr_ind = 'Y', start_date = today, end_date = 9999-12-31).

T00208_20260623150702	T00208	C00007	2026-05-31	2026-06-04	2026-06-23 15:06:56	LCK_20260623_150655_XAJIOY	180353c2dd58b9923874fc5ad0b17863	2026-06-23	2026-06-25	N
T00208_20260626131808	T00208	C00007	MAX	MAX	2026-06-26 13:17:59	LCK_20260626_131758_XAJIOY	e36fc55d1aec31c80105266fca01bf5	2026-06-26	9999-12-31	Y
T00209_20260623150702	T00209	C00027	2021-10-17	2021-10-22	2026-06-23 15:06:56	LCK_20260623_150655_XAJIOY	7e8a623fc6429fca19f806fb709074b	2026-06-23	9999-12-31	Y

Fig 22 — Trainer T00208: the earlier row is expired with end_date = 2026-06-25 and curr_ind = 'N', while the new version for the same trainer_id starts on 2026-06-26 with curr_ind = 'Y' and an open-ended end_date of 9999-12-31 — the two dates are exactly one day apart, so there is no gap or overlap in history.

Across the table, curr_ind = 'Y' rows consistently carry the open-ended end_date 9999-12-31, while curr_ind = 'N' rows carry a real, past end_date — the sentinel value only ever appears on the currently active version of a record.

9999-12-31	Y
2026-07-04	N
9999-12-31	Y
9999-12-31	Y
9999-12-31	Y
9999-12-31	Y
9999-12-31	Y

Fig 23 — end_date / curr_ind pairs from base_trainer: one expired row (real end_date, 'N') alongside several currently active rows (9999-12-31, 'Y').

4.2.3 Single Active-Version Check

Grouping base_trainer by trainer_id, filtered to curr_ind = 'Y', and keeping only groups with more than one row, should return zero rows if the single-active-version rule holds. An empty result set is the expected, successful outcome — it means no business key was found with two simultaneously active versions.

```
120
121 • SELECT trainer_id,
122      COUNT(*)
123      FROM base_trainer
124      WHERE curr_ind='Y'
125      GROUP BY trainer_id
126      HAVING COUNT(*)>1;
```

Result Grid



Filter Rows:

	trainer_id	COUNT(*)
--	------------	----------

Result 12



Fig 24 — `SELECT trainer_id, COUNT(*) FROM base_trainer WHERE curr_ind='Y' GROUP BY trainer_id HAVING COUNT(*)>1` — empty result grid, confirming no `trainer_id` has more than one active row.

5. Testing

5.1 Landing Layer Test Results

The following checks validate the Landing Layer behaviour demonstrated in Section 2.2

Test	What It Verifies	Result
Skip-if-missing test	Entity is skipped gracefully when no source file is present, without halting the run.	PASS
Empty file handling test	Empty files are discarded and not inserted into the landing table.	PASS
Unsupported file type test	Non CSV/XLSX/JSON files (e.g. .md) are rejected and logged as failed without stopping the pipeline.	PASS
Chronological order test	Files are picked up in ascending filename-date order (oldest first).	PASS
Load Key uniqueness test	Every execution generates a distinct Load Key, recorded once in the Load Control table.	PASS
Audit completeness test	Every processed file has a corresponding audit row with status, timestamp, path and row count.	PASS
Metadata-only append test	Landing rows retain raw source values; only Load Key and Ingestion Timestamp are added.	PASS
Idempotent re-run test	Re-running the pipeline against already-processed source files does not reload or duplicate their landing rows.	PASS

5.2 Staging Layer Test Results

The following checks validate the Staging Layer behaviour demonstrated in Section 3.2

Test	What It Verifies	Result
Full transfer test	All records from Landing are carried into the corresponding Staging tables on first load.	PASS
Checksum generation test	A checksum is computed for every record to support change detection against Base.	PASS
Change-flag assignment test	Records are correctly flagged I (new), U (changed) or N (no change).	PASS
Duplicate suppression test	Duplicate records (same business key/checksum) are identified and not double-counted for update.	PASS
Date validation test	Invalid dates and inconsistent ranges (Start_Date > End_Date) are detected and nulled.	PASS
Standardization test	Names are capitalized, gender values normalized, and date formats made consistent.	PASS
Base-count reconciliation test	Staging row count for a first load matches the corresponding Landing row count.	PASS
Mandatory field validation test	Required IDs and business attributes are validated before further processing.	PASS
Post-load truncation test	Staging tables are truncated only after a successful Base load.	PASS

5.3 Base Layer Test Results

The following checks validate the Base Layer behaviour demonstrated in Section 4.2.

Test	What It Verifies	Result
New-record insertion test	A new business key from Staging is inserted into Base with curr_ind = 'Y', start_date = today and end_date = 9999-12-31.	PASS
SCD Type 2 expiry test	An updated business key's previous row is expired (curr_ind = 'N', end_date = new start_date - 1) before the new version is inserted.	PASS
Single active-version test	No business key has more than one row with curr_ind = 'Y' at the same time.	PASS
End-date continuity test	The expired row's end_date is exactly one day before the new row's start_date, with no gap or overlap.	PASS
Change-flag N skip test	Rows carrying change_flag = N from Staging are not inserted or modified in Base.	PASS
Truncate-after-success test	A Staging table is cleared only after its Base load for that run completes without error.	PASS

5.4 Cross-Layer Testing (Landing → Staging → Base)

These are the standard pipeline-wide checks planned for this project.

Test	What It Verifies	Result
A → B Testing (Landing → Staging)	Left anti-join to confirm no keys are lost between Landing and Staging on first load.	PASS
A → B Testing (Staging → Base)	Left anti-join to confirm every I/U-flagged Staging record lands in Base.	PASS
A → B Testing (Landing → Base)	End-to-end anti-join across all three layers.	PASS
Row Count Testing (Landing vs Staging)	Row counts match between Landing and Staging on first load.	PASS
Row Count Testing (Staging vs Base)	Staging count immediately before truncation compared against newly-inserted Base rows for the same batch.	PASS
Duplicate Key Testing (Staging)	No duplicate business key + load batch combination in Staging.	PASS
Duplicate Key Testing (Base)	No duplicate active (Current_Ind = 'Y') record per business key in Base.	PASS
Pipeline Audit Test	Load Control and Audit tables correctly record execution details for every ETL stage.	PASS

6. Views & Power BI Analysis

6.1 Reporting View

A reporting view is built on top of the Base layer, joining Employee, Course/Enrollment and Assessment records into a single row per employee-course combination. It exposes employee profile fields (name, email, primary skill, management level) alongside course status, score and grade, and the final assessment outcome — giving Power BI a single, ready-to-use reporting dataset.

emp_id	first_nm	last_nm	email	primary_skill	management_level	course_id	course_nm	course_status	score_prct	grade	assessment_status
E01002	Vernon	Harrington	morankenneth@example.net	Technical_Product_Knowledge	12	C00027	Technical Product Management	Completed	52	P2	Passed
E01002	Vernon	Harrington	morankenneth@example.net	Technical_Product_Knowledge	12	C00018	Application Security Foundations	Cancelled	NA	NA	Cancelled
E01003	Russell	Franco	shane82@example.com	Budgeting_Forecasting	11	C00023	Budgeting And Forecasting	Completed	57	P2	Passed
E01003	Russell	Franco	shane82@example.com	Budgeting_Forecasting	11	C00006	Data Analysis With Python	In Progress	NA	NA	Not Appeared
E01003	Russell	Franco	shane82@example.com	Budgeting_Forecasting	11	C00009	Power BI For Business Analytics	In Progress	NA	NA	Not Appeared
E01004	Joseph	Anderson	tranvictor@example.com	Cloud Platform - Azure	10	C00020	Azure Cloud Fundamentals	Completed	52	P2	Passed
E01005	Kimberly	Richardson	smithkristina@example.net	Visualization - Power_BI	9	C00022	Risk Management Fundamentals	Completed	89	P4	Passed
E01005	Kimberly	Richardson	smithkristina@example.net	Visualization - Power_BI	9	C00023	Budgeting And Forecasting	Enrolled	NA	NA	Not Appeared
E01006	Meghan	Woodward	qsnov@example.net	Version Control - Git	11	C00029	Campaign Management Essentials	Completed	60	P2	Passed
E01006	Meghan	Woodward	qsnov@example.net	Version Control - Git	11	C00006	Data Analysis With Python	Completed	73	P3	Passed
E01008	Lisa	Lopez	jacksonjohn@example.com	Cloud Platform - Azure	12	C00020	Azure Cloud Fundamentals	Cancelled	NA	NA	Cancelled
E01008	Lisa	Lopez	jacksonjohn@example.com	Cloud Platform - Azure	12	C00047	Sap Fico Complete Training	Completed	78	P3	Passed
E01008	Lisa	Lopez	jacksonjohn@example.com	Cloud Platform - Azure	12	C00034	Seo Mastery Course	Enrolled	NA	NA	Not Appeared
E01010	Mason	Matthews	edward05@example.org	Server Admin - Windows	11	C00016	Windows Server Administration	Cancelled	NA	NA	Cancelled
E01011	Jennifer	Durham	fschultz@example.net	Payroll_Management	12	C00033	Payroll Management Certification	Completed	56	P2	Passed
E01012	Christina	Johnson	johndennis@example.net	Financial_Reporting	11	C00030	Financial Reporting And Analysis	In Progress	NA	NA	Not Appeared
E01012	Christina	Johnson	johndennis@example.net	Financial_Reporting	11	C00010	Selenium Webdriver Automation	In Progress	NA	NA	Not Appeared
E01012	Christina	Johnson	johndennis@example.net	Financial_Reporting	11	C00025	Product Roadmapping And Strat...	In Progress	NA	NA	Not Appeared

Fig 26 — reporting view output: one row per employee-course pairing with course_status, score_prct, grade and assessment_status columns ready for Power BI consumption.

6.2 Power BI Analysis

Two visuals built on the reporting view give a quick read on workforce training outcomes.

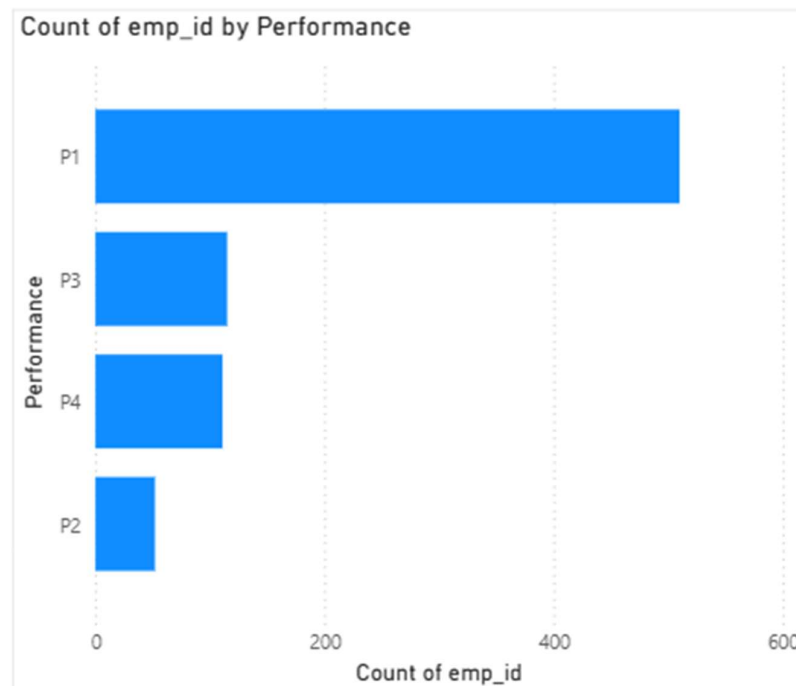


Fig 27 — count of employees by Performance band (P1–P4): the majority of employees fall into the P1 band, with smaller counts in P3, P4 and P2.

Assessment completion is summarised separately by outcome:

Count of emp_id by assessment_status

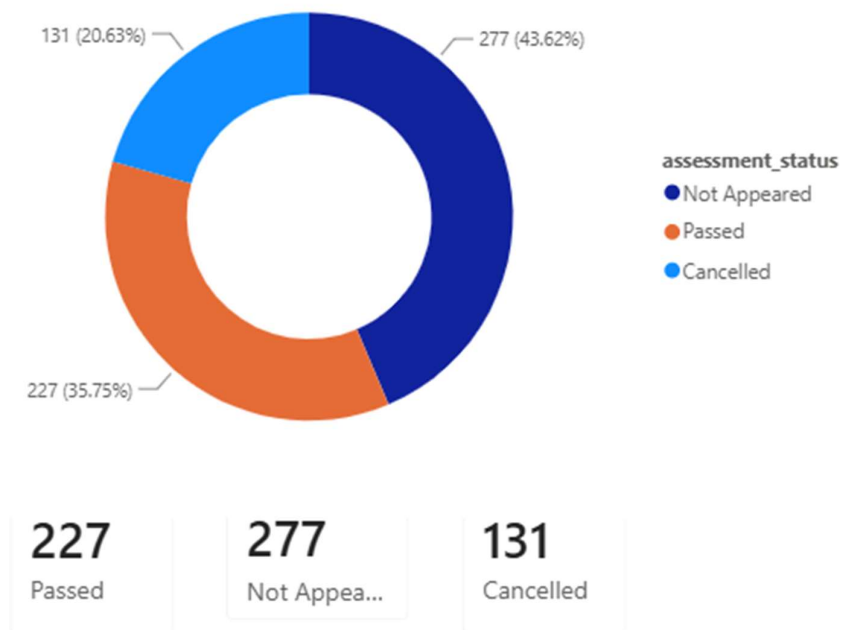


Fig 28 — count of employees by assessment_status: 277 Not Appeared (43.62%), 227 Passed (35.75%) and 131 Cancelled (20.63%).

7. Conclusion

This project successfully implements an end-to-end ETL pipeline for the Learning and Training Management (LTM) system using a three-layer architecture consisting of Landing, Staging and Base layers.

The Landing layer reliably ingests data from multiple source formats, while the centralized Load Control and Audit framework provides complete execution tracking across the Landing, Staging, and Base layers. Every ETL execution is uniquely identified through a Load Key, enabling full traceability, monitoring, and debugging of the pipeline. The Staging layer performs data cleansing, validation, standardization, checksum generation and business-rule enforcement before identifying new, updated and unchanged records using SCD Type 2 logic. The Base layer maintains historical versions of records while preserving data integrity through version control using curr_ind, start_date and end_date.

Comprehensive testing was performed across all layers to validate data movement, transformations, duplicate handling, mandatory field validation, historical versioning and cross-layer reconciliation. The reporting view and Power BI dashboards demonstrate how the processed data can be transformed into meaningful business insights for training and assessment reporting.

Overall, the project demonstrates the implementation of a reliable ETL workflow with proper data quality checks, historical tracking, auditability and reporting support, following industry-standard data engineering practices.